Informe de uso de IA – Laboratorio 8 GTICS

Nov 5, 2026 · @lu

En una sesión de unas 1 h 10 min se implementaron con apoyo de IA la edición y la eliminación de posts con control de permisos, y se resolvió un error de despliegue en Docker. El envío de solicitudes de amistad queda pendiente.

Datos generales

Campo	Detalle
Curso	GTICS
Actividad	Laboratorio 8
Estudiante	a20233424@pucp.edu.pe
Fecha y horario	5 de noviembre de 2026, 20:51 a 21:59 (hora de Lima)
Herramienta de IA	Claude (Anthropic), modelo Claude Opus 5.5
Repositorio base	lewisrengifo/gtics-lab8-base (Spring Boot 2.7, Thymeleaf, Spring Security, Spring Session JDBC, MySQL)
Proyecto propio desplegado	Lab8 (Spring Boot 3.5.6) en un contenedor Docker

Objetivo de la sesión

Usar el repositorio base como punto de partida para completar tres funcionalidades del blog. Durante la sesión se sumó un cuarto tema: un error al desplegar el proyecto Lab8 en Docker.

1. Edición de post.
2. Eliminación de post.
3. Envío de solicitudes de amistad (solo el envío, no la aceptación).
4. Despliegue del proyecto Lab8 en un contenedor Docker conectado a MySQL.

Al final de la sesión quedaron hechas la 1, la 2 y la 4. La 3 no se empezó.

Cronología de la interacción

La sesión tuvo 11 intercambios. Las horas son las de cada mensaje del estudiante.

Hora	Pedido o pregunta del estudiante	Resultado
20:54	Revisar el repositorio base antes de construir sobre él	La IA clonó el repo, leyó todo el código y entregó un diagnóstico: errores en el formulario y controller de posts, navbar roto, script SQL que no carga, y una ruta de borrado sin proteger
20:56	Implementar la edición de post (solo el autor) y hacer un diagrama de permisos y sesiones	Código corregido y parche `edicion-post.patch`. Diagrama en `docs/permisos.md`. La versión en FigJam quedó pendiente de elegir un plan
21:03	¿Qué pasa si un usuario cambia el `id` en `/post/edit`?	Explicación de los 5 casos posibles y del ataque IDOR
21:15	¿Qué hace `GRANT ALL PRIVILEGES ON Lab5.*` y por qué es mejor que root?	Explicación del principio de mínimo privilegio y del alcance del permiso
21:31	¿Qué parte del código evitó la edición de un post ajeno al escribir la URL a mano?	Se señaló el método `esAutor` en `PostController` y por qué las otras capas no bastaban
21:51	El contenedor Docker se encendía y se apagaba (log adjunto)	Diagnóstico: `Access denied for user 'lab8_app'@'localhost'`. Pasos para revisar y corregir el usuario de MySQL
21:53	El estudiante compartió su propia solución en SQL	La IA la validó y señaló tres puntos a confirmar
21:56	La regla dice `/post/delete` pero el controller usa `/post/borrar`: ¿queda protegida?	No: la ruta caía en `permitAll()`. También se detectó el problema de la barra final con `antMatchers`
21:58	Decisión: pueden borrar el admin y el autor	Eliminación implementada. Parches `eliminacion-post.patch` y `edicion-y-eliminacion-completo.patch`
21:58	¿Por qué ya no existe `openjdk:17-jdk-slim` y qué es `eclipse-temurin`?	Explicación con fuentes y un `Dockerfile` corregido
21:59	Generar este informe	Este documento

Cambios de código realizados

El cambio central: editar y borrar un post se decide en el servidor, comparando el autor guardado en la BD con el usuario de la sesión. Ocultar el botón en la vista no basta.

Archivo	Cambio	Motivo
`PostController.java`	`GET /post/create`, `GET /post/edit`, `POST /post/guardar` y `POST /post/borrar` reescritos, con el método `esAutor`	Solo el autor edita; el admin o el autor borran. La verificación se repite en el POST porque el `id` del formulario se puede manipular
`PostController.java`	Al editar se carga el post de la BD y solo se copian título, resumen, contenido y publicado	El autor nunca sale del formulario
`WebSecurityConfig.java`	Se quitó la regla `/post/delete` (ruta inexistente), se protegieron `/post/guardar` y `/post/borrar`, y se cambió a `mvcMatchers`	Antes cualquiera, sin sesión, podía borrar; con `antMatchers`, `/post/borrar/` se saltaba la regla
`Post.java`	`@NotBlank` y `@Size` en título y resumen	Validación del formulario
`post/form.html`	Campos corregidos (`id_Post`), checkbox "Publicado", mensajes de error, navbar	El formulario lanzaba error por nombres de campo inexistentes
`post/lista.html`	Botón Editar solo para el autor; Borrar como formulario POST (con CSRF) para admin o autor; mensajes de éxito y error	Botones invisibles y error para visitantes anónimos
`base.html`	Nombre del usuario corregido y logout como POST	El navbar rompía todas las páginas con sesión iniciada
`LoginController.java`	Ruta `GET /loginForm`	Spring Security redirigía a una página que no existía (404)

Archivo	Cambio	Motivo
`Lab5_script.sql`	`--- Roles` y `--- Users` cambiados a `--`	MySQL no aceptaba esos comentarios y no cargaba roles ni usuarios
`docs/permisos.md`	Diagrama Mermaid y tablas de quién puede hacer qué	Documentación de permisos y sesiones

Permisos resultantes sobre un post:

Acción	Anónimo	Autor (`user`)	Otro `user`	`admin` no autor
Ver lista	Sí	Sí	Sí	Sí
Crear	No	Sí	Sí	Sí
Editar	No	Sí	No	No
Borrar	No	Sí	No	Sí

Problemas de despliegue en Docker

Hubo dos problemas; el estudiante resolvió el primero con su propio script SQL tras el diagnóstico de la IA.

Problema	Causa	Solución
El contenedor `lab8` quedaba en `Exited (1)` al arrancar	`Access denied for user 'lab8_app'@'localhost'` : MySQL no reconocía esa combinación de usuario, contraseña y host. Spring Session no podía crear sus tablas y la app se caía	El estudiante creó el usuario `lab8_app` para `localhost` y `%` , fijó la contraseña y le dio `GRANT ALL PRIVILEGES` sobre la base `Lab5` . La IA confirmó que la solución era correcta
El `Dockerfile` fallaba con `openjdk:17-jdk-slim: not found`	La imagen `openjdk` de Docker Hub está obsoleta desde julio de 2022 y en 2025 se borraron casi todas sus etiquetas (PR #550)	Cambiar la imagen base a `eclipse-temurin:17-jre` , mantenida por Eclipse Adoptium

Recomendaciones que dejó la IA: no guardar la contraseña en `application.properties` ni en el `Dockerfile`, sino pasarla con `-e SPRING_DATASOURCE_PASSWORD`, y usar una más fuerte si la app se expone fuera de la máquina.

Conceptos aprendidos

Las preguntas del estudiante se centraron en seguridad y despliegue, y cubrieron ocho conceptos.

Concepto	Qué se entendió	Dónde se aplicó
Tres capas de control	Spring Security revisa el rol; el controller revisa si el recurso es tuyo; la vista solo oculta botones	Edición y eliminación de post
IDOR (*Insecure Direct Object Reference*)	Cambiar un `id` en la URL o en un campo oculto para llegar a un recurso ajeno; se evita validando en el servidor	`esAutor` en `GET /post/edit` y `POST /post/guardar`
Orden de reglas en Spring Security	Se aplica la primera regla que coincide; una ruta sin regla cae en `anyRequest().permitAll()`	La regla `/post/delete` no protegía `/post/borrar`
`antMatchers` vs `mvcMatchers`	En Boot 2.7, `antMatchers` no reconoce `/ruta/` con barra final y la regla se puede saltar	Reglas de `WebSecurityConfig`
CSRF	Solo protege `POST`, `PUT` y `DELETE`; borrar con `GET` deja la acción expuesta a otras páginas	Borrado como formulario POST
Mínimo privilegio en MySQL	`GRANT ALL PRIVILEGES ON Lab5.*` limita el daño a una base, frente a root, que controla todo el servidor	Usuario `lab8_app` del contenedor
Usuarios de MySQL por host	`'usuario'@'localhost'` y `'usuario'@'%'` son cuentas distintas	Error `Access denied` del contenedor
Imágenes base de Java	`openjdk` ya no recibe parches; `eclipse-temurin` sí, y su variante `-jre` es más liviana	`Dockerfile` del Lab8

Trabajo de la IA y del estudiante

La IA escribió el código de edición y eliminación; el estudiante definió las reglas de permisos, desplegó la app y resolvió el acceso a MySQL.

Realizado por la IA	Realizado o decidido por el estudiante
Revisión del repositorio y diagnóstico de errores	Elegir el repositorio base y las funcionalidades a construir
Código de edición y eliminación, entregado como parches	Decidir que solo el autor edita y que el admin y el autor borran
Diagrama y documentación de permisos (`docs/permisos.md`)	Pedir el diagrama e incluir sesiones y permisos
Diagnóstico del log de Docker	Construir la imagen, ejecutar el contenedor y obtener los logs
Validación del script SQL	Escribir y ejecutar el script que creó el usuario `lab8_app`
Respuestas conceptuales (IDOR, GRANT, reglas de seguridad, imágenes de Java)	Plantear las preguntas a partir de lo que observó al probar

Verificación. La IA no pudo compilar ni ejecutar el proyecto en su entorno; el código se revisó solo por lectura. La prueba real queda a cargo del estudiante al aplicar los parches.

Pendientes:

- [ ] Aplicar los parches, compilar y probar editar y borrar con un autor, otro usuario y el admin
- [ ] Implementar el envío de solicitudes de amistad (tabla nueva, entidad, repositorio y `POST` con validaciones)
- [ ] Corregir el módulo de usuarios (rutas `/usuario/...` mal escritas y `/user` sin controlador)
- [ ] Confirmar que el contenedor `lab8` queda en estado `Up` con la nueva imagen `eclipse-temurin`
- [ ] Generar el diagrama en FigJam (falta elegir el plan) o usar el de `docs/permisos.md`

Declaración de uso de IA

Declaro que en este laboratorio usé Claude (Anthropic) como asistente para revisar código, generar las correcciones descritas y resolver dudas conceptuales. Las decisiones de diseño (quién puede editar y borrar), el despliegue en Docker y la configuración de

MySQL fueron mías. Revisé las explicaciones y me comprometo a probar el código antes de entregarlo; puedo explicar cada cambio si se me solicita.

Firma: _____

Fuentes

- Repositorio base lewisrengifo/gtics-lab8-base
- docker-library/openjdk, PR #550
- Red Hat Developer: Updating Docker Hub's OpenJDK image